

Informe Técnico Ejecutivo - Evaluación Final Transversal

Netflix User Behavior - Churn Analytics + Machine Learning

SCY1101 Programación para la Ciencia de Datos

Integrantes: Johann Carrasco, Pablo Daza y Ricardo Ruiz.

Versión del proyecto: EFT con ETL, API, dashboard, Machine Learning, Docker y CI/CD.

1. Executive summary

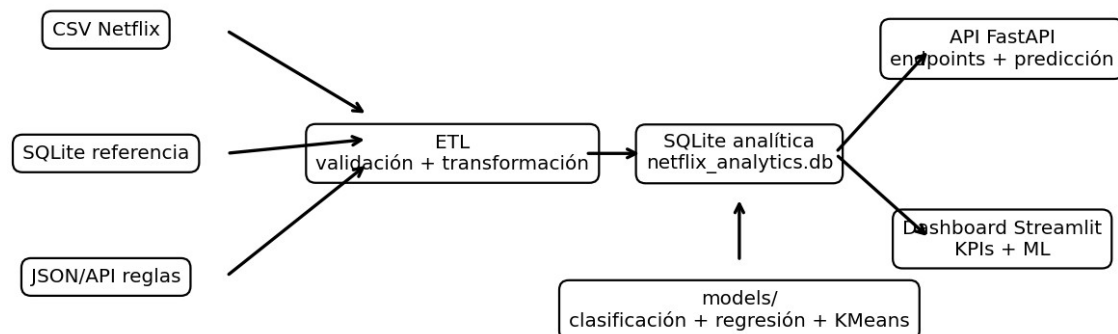
El proyecto consolida la solución end-to-end desarrollada en la Evaluación 3 y la eleva con una capa de Machine Learning. El problema de negocio es identificar patrones asociados al abandono de usuarios de Netflix (churn) y entregar información accionable para decisiones de retención. La solución integra múltiples fuentes de datos, ejecuta un pipeline ETL, almacena datos procesados en SQLite, entrena modelos predictivos, expone resultados mediante API REST y presenta KPIs en un dashboard interactivo.

2. KPIs principales

Indicador	Valor
Usuarios totales	50000
Usuarios churn	9964
Usuarios no churn	40036
Churn rate	19.93%
Tiempo promedio de visualización	154.95 minutos
Ingreso activo estimado	US\$ 493,787

3. Arquitectura técnica

La arquitectura sigue el flujo: CSV principal + SQLite de referencia + reglas JSON/API -> ETL -> SQLite analítica -> API, dashboard y modelos ML. Esta estructura evita trabajar directamente con el CSV en la visualización y permite que API y dashboard consuman datos procesados, validados y reproducibles.



4. Pipeline ETL

El pipeline se ejecuta desde `etl/run_etl.py`. Primero extrae el CSV principal, la base SQLite de referencia y las reglas de negocio en formato JSON/API. Luego valida columnas obligatorias, duplicados, nulos críticos y rangos numéricos. Después aplica transformaciones de negocio, crea variables como `churned_binary`, `nivel_actividad`, `riesgo_login`, `engagement_score` y `churn_risk_segment`. Finalmente carga el dataset limpio y las tablas agregadas en `data/processed/netflix_analytics.db`.

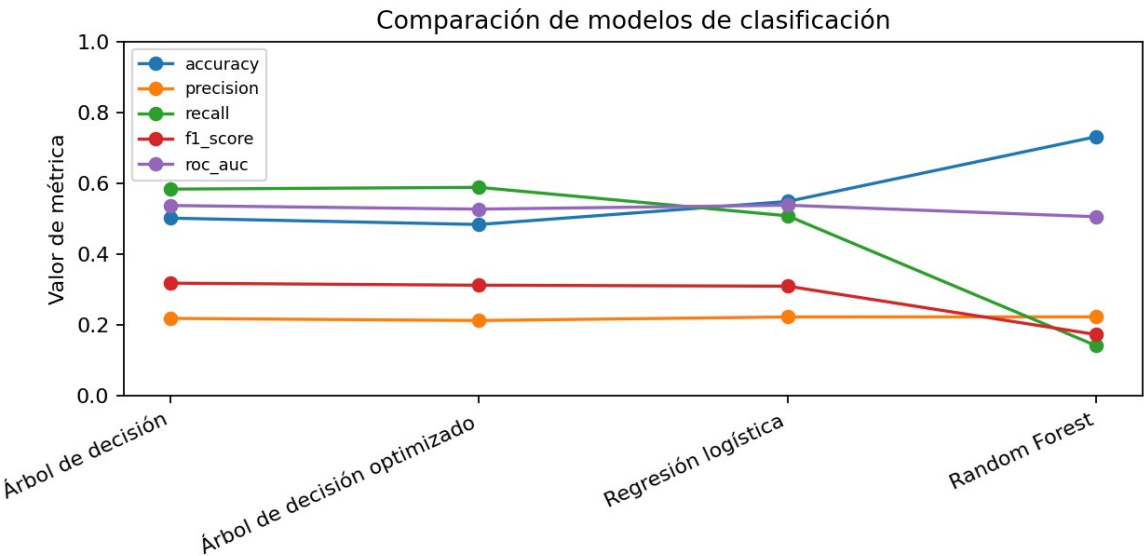
5. Capa de Machine Learning

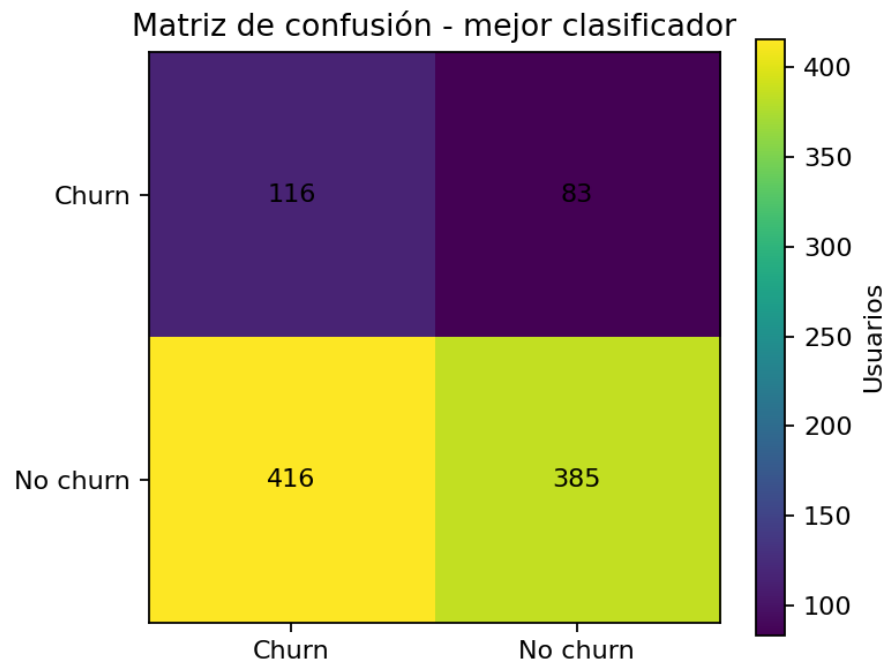
La carpeta `models/` incorpora el entrenamiento de modelos de clasificación, regresión y segmentación. Los modelos se implementan con Scikit-learn usando pipelines con preprocesamiento, imputación, escalado y codificación `OneHotEncoder`. Los artefactos se serializan como `.pkl` para ser utilizados por la API.

5.1 Modelos de clasificación

El objetivo de clasificación es predecir la variable `churned_binary`. Se comparan Regresión Logística, Árbol de Decisión, Random Forest y un Árbol de Decisión optimizado con `RandomizedSearchCV`. La métrica de mayor interés es F1-score porque equilibra precisión y recall en un problema donde detectar usuarios churn es más relevante que solo maximizar accuracy.

modelo	accuracy	precision	recall	f1_score	roc_auc
Árbol de decisión	0.5010	0.2180	0.5829	0.3174	0.5366
Árbol de decisión optimizado	0.4830	0.2120	0.5879	0.3116	0.5264
Regresión logística	0.5480	0.2220	0.5075	0.3089	0.5375
Random Forest	0.7310	0.2222	0.1407	0.1723	0.5049

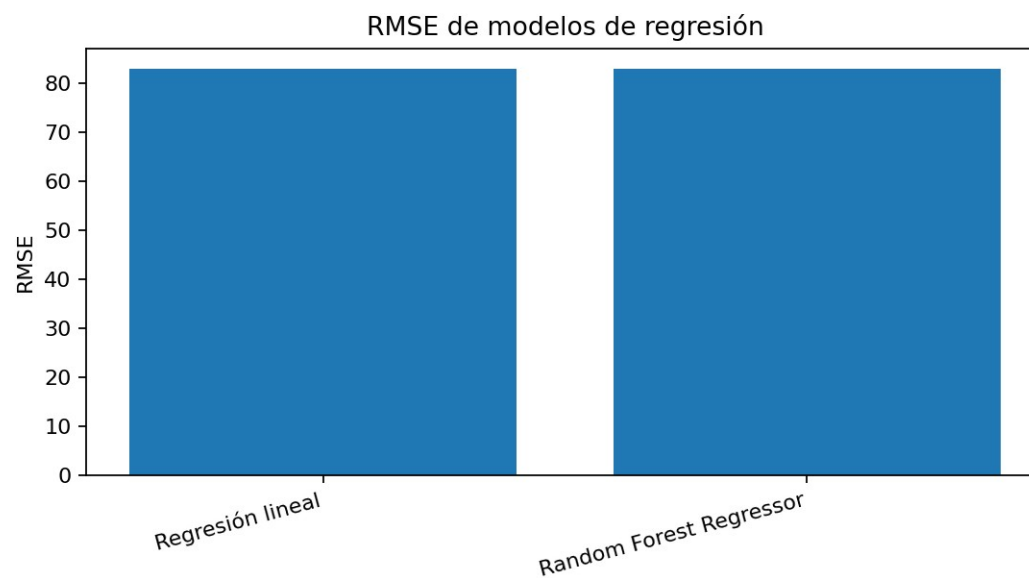




5.2 Modelos de regresión

El objetivo de regresión es estimar avg_watch_time_minutes. Se comparan Regresión Lineal y Random Forest Regressor. Las métricas utilizadas son MAE, RMSE y R2. Estas métricas permiten evaluar error promedio, penalización de errores grandes y capacidad explicativa del modelo.

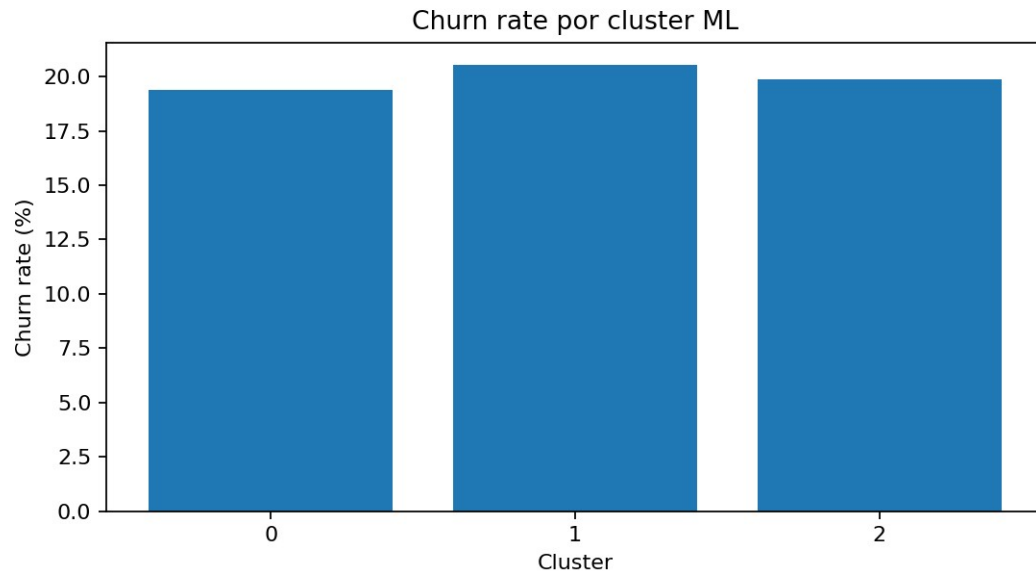
modelo	mae	rmse	r2
Regresión lineal	71.4599	82.8812	-0.0185
Random Forest Regressor	71.4384	82.9429	-0.0200



5.3 Segmentación no supervisada

Además, se aplica KMeans para segmentar usuarios según actividad, engagement, días desde último login y valor de suscripción. El objetivo es generar grupos operativos que faciliten priorizar campañas de retención y análisis de comportamiento.

cluster_ml	usuarios	churn_rate	avg_engagement	avg_watch_time	avg_days_since_login
0.0000	1961.0000	0.1938	0.5994	155.8440	29.2030
1.0000	1831.0000	0.2054	0.3505	155.1316	28.8050
2.0000	1208.0000	0.1987	0.5712	155.4702	30.5240



6. API REST

La API está implementada con FastAPI en `api/main.py`. En la versión EFT se agregan endpoints para consultar métricas de modelos y realizar predicciones por usuario. Los endpoints más importantes son `/health`, `/summary`, `/model-metrics`, `/confusion-matrix`, `/predict-churn/{user_id}` y `/predict-watch-time/{user_id}`.

7. Dashboard interactivo

El dashboard está implementado con Streamlit en `dashboards/app.py`. Mantiene las vistas ejecutiva, técnica y operativa, y agrega una pestaña de Modelos ML. Esta pestaña permite revisar comparación de métricas, matriz de confusión, resultados de regresión y clusters de usuarios.

8. Docker y CI/CD

El proyecto mantiene Dockerfiles y `docker-compose.yml` para levantar API, ETL, entrenamiento ML y dashboard de forma reproducible. Además, se incorpora un workflow de GitHub Actions en `.github/workflows/python-ci.yml` que instala dependencias, ejecuta ETL, entrena modelos y corre pruebas automáticas con `pytest`.

9. Pruebas y reproducibilidad

La carpeta tests/ contiene pruebas básicas para validar componentes clave del ETL y la API. El flujo recomendado para reproducir el sistema es: instalar dependencias, ejecutar ETL, entrenar modelos, levantar API y abrir dashboard.

10. Valor de negocio

El valor principal del proyecto es convertir datos de comportamiento en información útil para retención. La combinación de dashboard, segmentación y predicción permite identificar usuarios con mayor riesgo, priorizar campañas y monitorear indicadores como churn rate, engagement y tiempo de visualización.

11. Comandos principales

```
python -m pip install -r requirements.txt
python etl/run_etl.py
python models/train_models.py
python -m uvicorn api.main:app --reload
python -m streamlit run dashboards/app.py
pytest -q
docker compose up --build
```

12. Conclusión

La solución final cumple con un enfoque profesional de ciencia de datos: integra fuentes, valida y transforma datos, entrena modelos predictivos, expone resultados por API, visualiza KPIs en dashboard, incorpora Docker y evidencia CI/CD. El proyecto queda preparado para ser defendido desde una perspectiva técnica y de negocio.